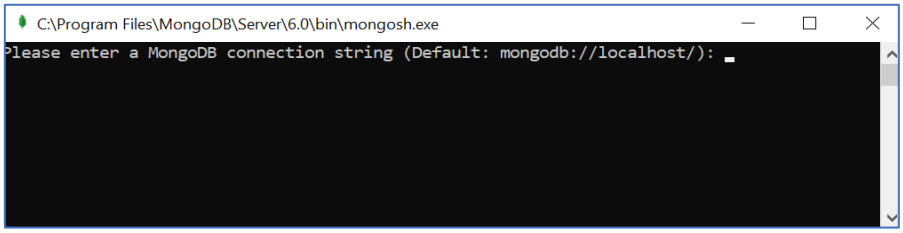
Revisar el ambiente de trabajo a través de la Shell de MongoDB

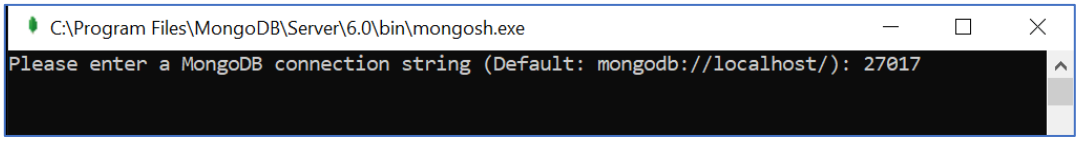
Paso 1: Ir al directorio de los archivos ejecutables de MongoDB. En el laboratorio se encuentra instalada la versión 5.0 y su ruta es C:\Program Files\MongoDB\Server\5.0\bin.

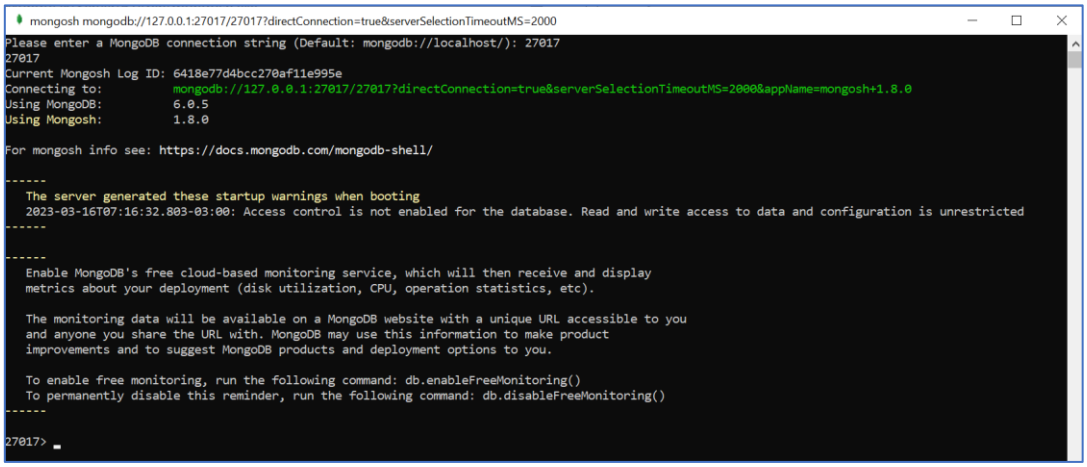
Como aclaración este manual utiliza la versión 6.0 por ende la ruta observada es C:\Program Files\MongoDB\Server\6.0\bin lo que en términos prácticos no tendremos problemas.

1. Ejecutar la Shell **mongosh.exe** para conectarse y operar con el servidor de base de datos



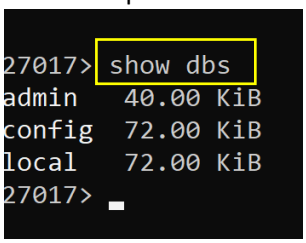
2. Ingresar el puerto de escucha del servidor mongodb. Esto es **27017** y presionar enter.





Se desplegará la base de datos por defecto: local 27017

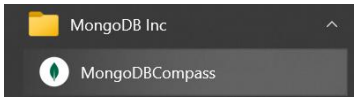
3. Consultar por todas las bases de datos del servidor utilizar el comando: **show dbs**



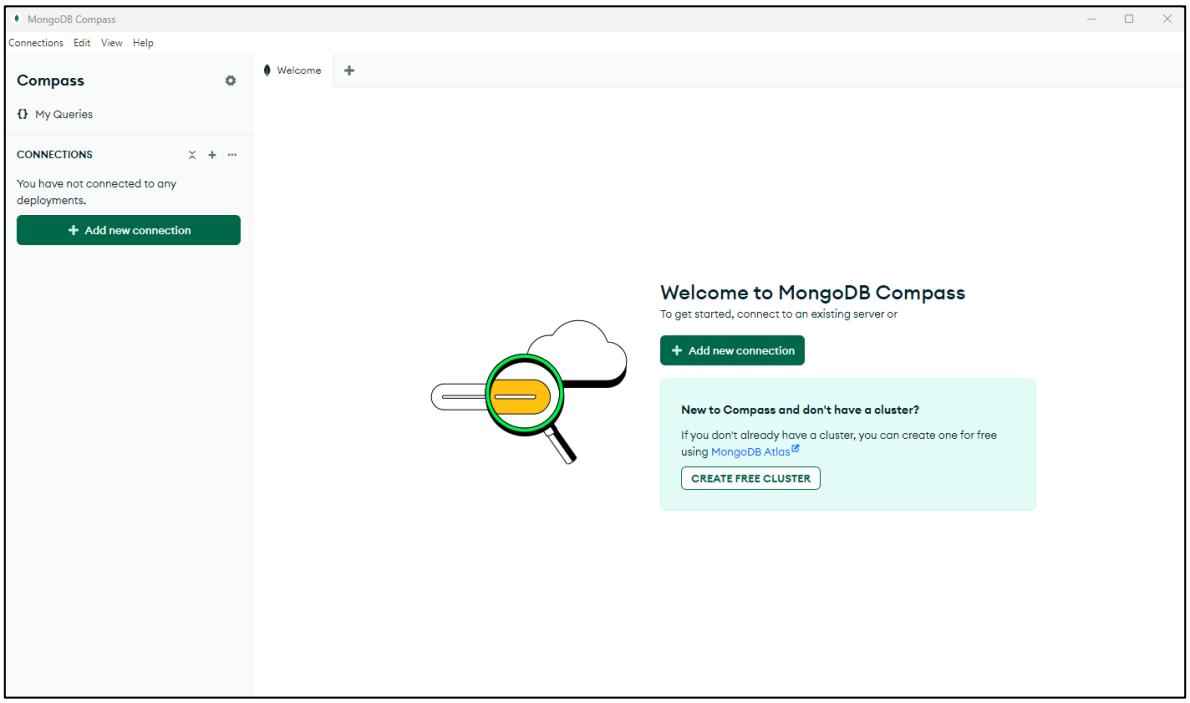
Nota para versiones inferiores de mongodb 6.0, la base por defecto es **test.** En este caso, **la base por defecto es 27017**

4. Revisar el cliente Mongo Compass y presionar conect para revisar las bases de datos: admin, config y local

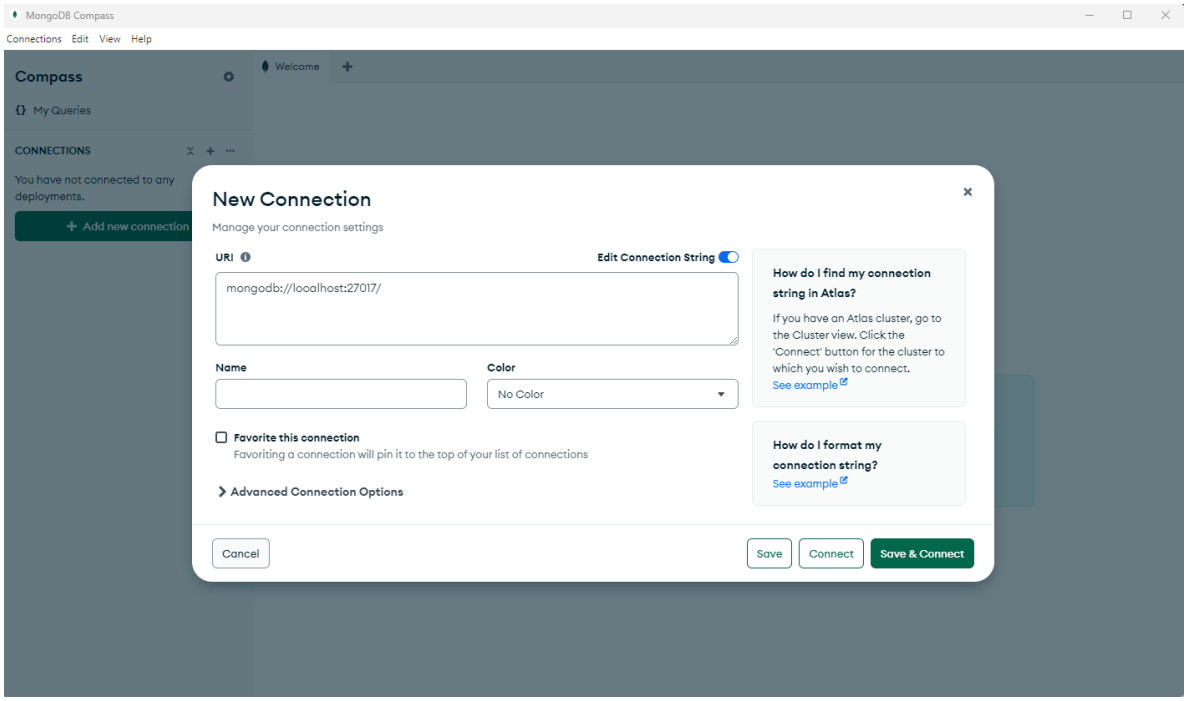
   Para conectarse con MongoDB Compas irse al menú de Windows y busca MongoCompas o lo buscas en el escritorio como se muestra en las siguientes imágenes y hacer doble click.



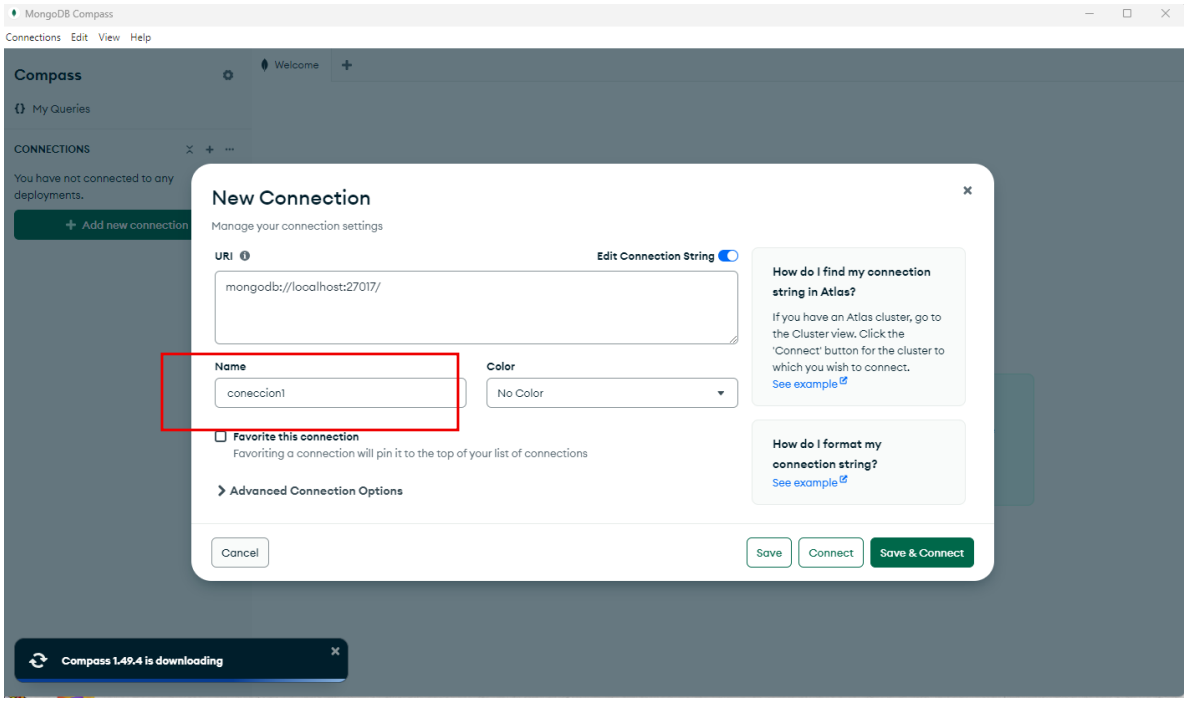
   En la versión 5.0 del laboratorio verás la siguiente interfaz y continúa presionando el botón Add New Connection.



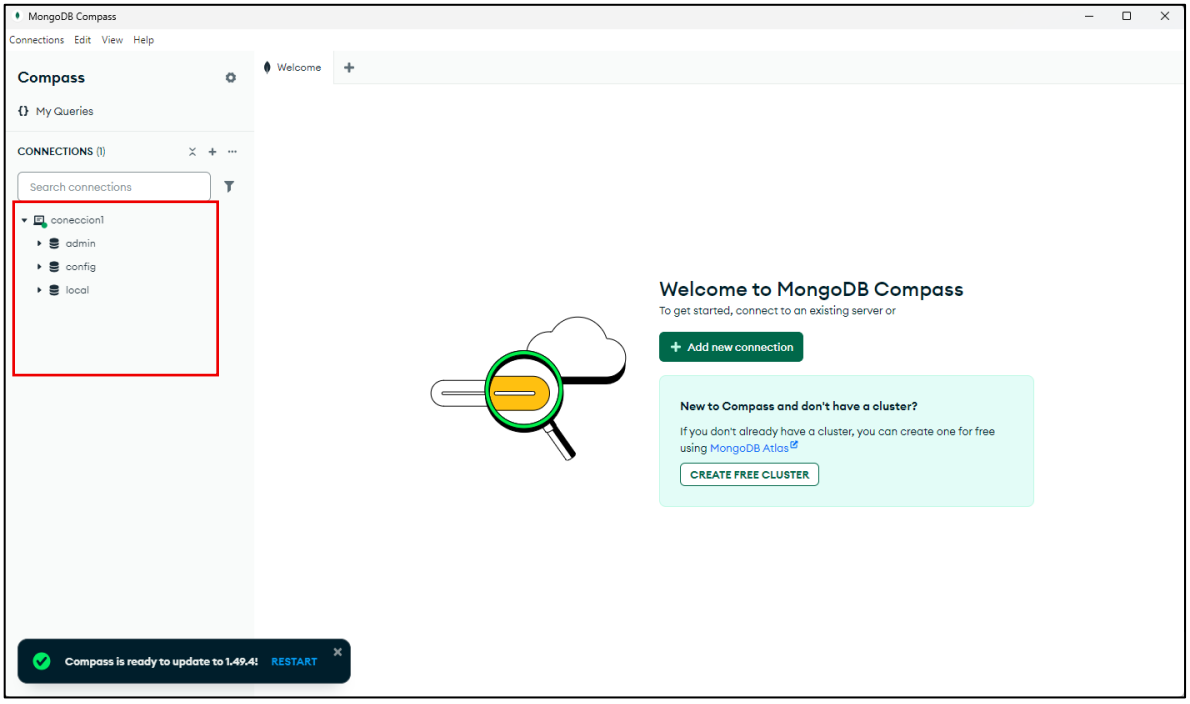
Ingresar nombre de la conexión en el campo Name como por ejemplo coneccion1



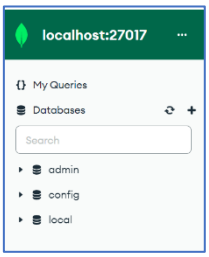
Presionar el botón Save & Connect



Observa las 3 bases de datos que se encuentran en MongoDB



En la versión 6.0 de MongoDB Compass aparece como:



5. Sigamos trabajando con la Shell. Ahora tenemos una conexión abierta con el servidor de base de datos mongodb. **Para crear una base de datos** debe realizar 2 pasos:
   a. **Paso 1**: Usar el siguiente comando: **use nombreDeLaBaseDatos**

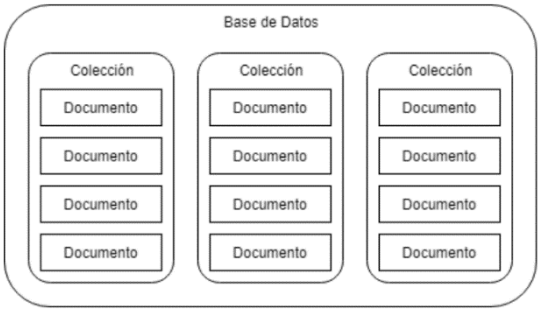
```
27017> use ejemplobd
switched to db ejemplobd
ejemplobd>
```

Nota: el comando **use** se utiliza para **crear una base de datos**, y también para **activar su utilización** en caso de que exista.

b. Aún no se ha creado por completo la base de datos con el paso 1. Esta se creará hasta que se ingrese algo a la base de datos. Validemos que aún no se puede ver la base de datos consultando a través del comando **shows dbs.** El resultado es que NO aparece la base de datos **ejemplobd**.

```
ejemplobd> show dbs
admin    40.00 KiB
config  108.00 KiB
local    72.00 KiB
ejemplobd>
```

c. **Paso 2**: Intentemos crear algo en la base de datos. Podemos crear una **colección** en la base de datos mongodb.



Nota:
La colección es una estructura de almacenamiento como las tablas son en el modelo relacional o lógico. El documento en MongoDB son como los registros(filas) en el modelo relacional.
Entonces, ¿Cómo creamos una colección?

Usar el comando
db.createCollection("usuarios")

```
ejemplobd> db.createCollection("usuarios")
{ ok: 1 }
```

d. Ahora, ¿Se creó la base de datos? Revisa y ejecuta **show dbs**

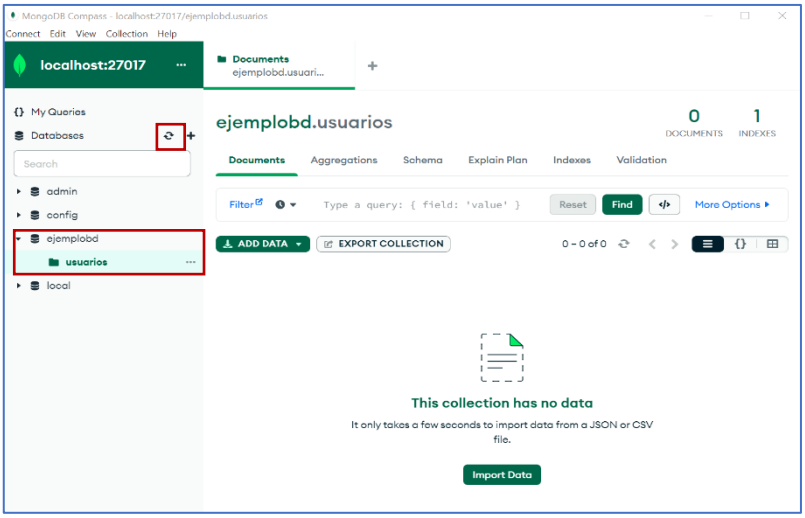
```
ejemplobd> show dbs
admin       40.00 KiB
config     108.00 KiB
ejemplobd    8.00 KiB
local       72.00 KiB
ejemplobd>
```

Efectivamente se creó la base de datos: **ejemplobd**

e. Para consultar por las colecciones usar el siguiente comando: **show collections**

```
ejemplobd> show collections
usuarios
ejemplobd>
```

f. Revisar la base de datos con Mongodb Compass. Refrescar la sección de Databases y abrir la base de datos ejemplobd. Podrás validar que la collection usuarios no posee datos.



6. Para revisar otros comandos. Podemos utilizar el comando **help**

```
ejemplobd> help

  Shell Help:

    use                                     Set current database
    show                                    'show databases'/'show dbs': Print a list of all available databases.
                                            'show collections'/'show tables': Print a list of all collections for current database.
                                            'show profile': Prints system.profile information.
                                            'show users': Print a list of all users for current database.
                                            'show roles': Print a list of all roles for current database.
                                            'show log <type>': log for current connection, if type is not set uses 'global'
                                            'show logs': Print all logs.

    exit                                    Quit the MongoDB shell with exit/exit()/.exit
    quit                                    Quit the MongoDB shell with quit/quit()
    Mongo                                   Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [optional])
    connect                                 Create a new connection and return the Database object. Usage: connect(URI, username [optional], password [optional])
    it                                      result of the last line evaluated; use to further iterate
    version                                 Shell version
    load                                    Loads and runs a JavaScript file into the current shell environment
ejemplobd>
    disableTelemetry                        Disables collection of anonymous usage data to improve the mongosh CLI
    passwordPrompt                          Prompts the user for a password
    sleep                                   Sleep for the specified number of milliseconds
    print                                   Prints the contents of an object to the output
    printjson                               Alias for print()
    convertShardKeyToHashed                 Returns the hashed value for the input using the same hashing function as a hashed index.
    cls                                     Clears the screen like console.clear()
ejemplobd>

  For more information on usage: https://docs.mongodb.com/manual/reference/method
ejemplobd>
```

Para mayor información de comando pueden consultar
https://docs.mongodb.com/manual/reference/method

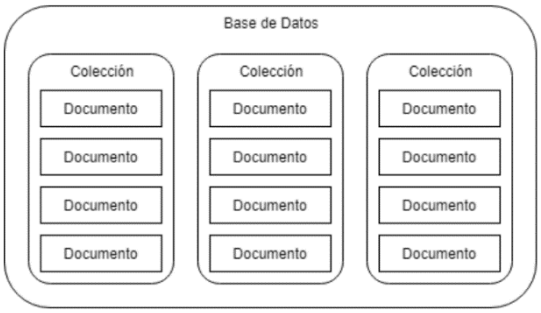
7. Para revisar otros comandos. Podemos utilizar la instrucción **db.help()**

```
ejemplobd> db.help()

  Database Class:

    getMongo                          Returns the current database connection
    getName                           Returns the name of the DB
    getCollectionNames                Returns an array containing the names of all collections in the current database.
    getCollectionInfos                Returns an array of documents with collection information, i.e. collection name and options, for the current database.
    runCommand                        Runs an arbitrary command on the database.
    adminCommand                      Runs an arbitrary command against the admin database.
    aggregate                         Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
    getSiblingDB                      Returns another database without modifying the db variable in the shell environment.
    getCollection                     Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
    dropDatabase                      Removes the current database, deleting the associated data files.
    createUser                        Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
    updateUser                        Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
    changeUserPassword                Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
    logout                            Ends the current authentication session. This function has no effect if the current session is not authenticated.
    dropUser                          Removes the user from the current database.
    dropAllUsers                      Removes all users from the current database.
    auth                              Allows a user to authenticate to the database from within the shell.
    grantRolesToUser                  Grants additional roles to a user.
    revokeRolesFromUser               Removes a one or more roles from a user on the current database.
    getUser                           Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
    getUsers                          Returns information for all the users in the database.
    createCollection                  Create new collection
    createEncryptedCollection         Creates a new collection with a list of encrypted fields each with unique and auto-created data encryption keys (DEKs). This is a utility function that internally utilises ClientEnryption.createEncryptedCollection.
    createView                        Create new view
    createRole                        Creates a new role.
    updateRole                        Updates the role's profile on the database on which you run the method. An update to a field completely replaces the previous field's values.
    dropRole                          Removes the role from the current database.
    dropAllRoles                      Removes all roles from the current database.
    grantRolesToRole                  Grants additional roles to a role.
    revokeRolesFromRole               Removes a one or more roles from a role on the current database.
    grantPrivilegesToRole             Grants additional privileges to a role.
    revokePrivilegesFromRole          Removes a one or more privileges from a role on the current database.
    getRole                           Returns role information for a specified role. Run this method on the role's database. The role must exist on the database on which the method runs.
    getRoles                          Returns information for all the roles in the database.
    currentOp                         Runs an aggregation using $currentOp operator. Returns a document that contains information on in-progress operations for the database instance. For further information, see $currentOp.
    killOp                            Calls the killOp command. Terminates an operation as specified by the operation ID. To find operations and their corresponding IDs, see $currentOp or db.currentOp().
    shutdownServer                    Calls the shutdown command. Shuts down the current mongod or mongos process cleanly and safely. You must issue the db.shutdownServer() operation against the admin database.
    fsyncLock                         Calls the fsync command. Forces the mongod to flush all pending write operations to disk and locks the entire mongod instance to prevent additional writes until the user releases the lock with a corresponding db.fsyncUnlock() command.
    fsyncUnlock                       Calls the fsyncUnlock command. Reduces the lock taken by db.fsyncLock() on a mongod instance by 1.
    version                           returns the db version. uses the buildInfo command
ejemplobd>
    isMaster                          Calls the isMaster command
    hello                             Calls the hello command
    serverBuildInfo                   returns the db serverBuildInfo. uses the buildInfo command
ejemplobd>
    stats                             returns the db stats. uses the dbStats command
    hostInfo                          Calls the hostInfo command
    serverCmdLineOpts                 returns the db serverCmdLineOpts. uses the getCmdLineOpts command
    rotateCertificates                Calls the rotateCertificates command
    printCollectionStats              Prints the collection.stats for each collection in the db.
    getFreeMonitoringStatus           Calls the getFreeMonitoringStatus command
    disableFreeMonitoring             returns the db disableFreeMonitoring. uses the setFreeMonitoring command
    enableFreeMonitoring              returns the db enableFreeMonitoring. uses the setFreeMonitoring command
    getProfilingStatus                returns the db getProfilingStatus. uses the profile command
    setProfilingLevel                 returns the db setProfilingLevel. uses the profile command
    setLogLevel                       returns the db setLogLevel. uses the setParameter command
    getLogComponents                  returns the db getLogComponents. uses the getParameter command
    cloneDatabase                     deprecated, non-functional
    cloneCollection                   deprecated, non-functional
    copyDatabase                      deprecated, non-functional
    commandHelp                       returns the db commandHelp. uses the passed in command with help: true
ejemplobd>
    getLastErrorObj                   Calls the getLastError command
    getLastError                      Calls the getLastError command
    printShardingStatus               Calls sh.status(verbose)
    printSecondaryReplicationInfo     Prints secondary replicaset information
    getReplicationInfo                Returns replication information
    printReplicationInfo              Formats sh.getReplicationInfo
    printSlaveReplicationInfo         DEPRECATED. Use db.printSecondaryReplicationInfo
    setSecondaryOk                    This method is deprecated. Use db.getMongo().setReadPref() instead
    watch                             Opens a change stream cursor on the database
    sql                               (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an experimental feature that may be subject to change in future releases.

ejemplobd>
```

8. Consultar por la base de datos actual con el comando **db**.

```
ejemplobd> db
ejemplobd
ejemplobd>
```

9. Insertar documento o documentos a la colección



Como se dijo anteriormente, la colección es una estructura de almacenamiento como las tablas son en el modelo relacional o lógico y el documento en MongoDB son como los registros en el modelo relacional.

Para insertar documentos en una colección, podemos realizarlo de varias formas.

a) Forma 1:

a. Crear documento.

Un documento tiene la siguiente estructura:

{dato, dato, dato, dato,…}

Recuerda que DATO es EL PAR → ATRIBUTO = VALOR

Entonces, podemos plantear dos documentos j y k

j={nombre: “Teresa”, país: “Chile”}

k={nombre: “Juan”, país: “Chile”}

```
ejemplobd> j={nombre:"Teresa", pais:"Chile"}
{ nombre: 'Teresa', pais: 'Chile' }
ejemplobd> k={nombre: "Juan", pais: "Chile"}
{ nombre: 'Juan', pais: 'Chile' }
ejemplobd>
```

b. Insertar los documentos a la colección **usuarios**

db.usuarios.insert(j)

db.usuarios.insert(k)

```
ejemplobd> db.usuarios.insert(j)
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany,
or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("641912df6f22ba679557492e") }
}
ejemplobd>

ejemplobd> db.usuarios.insert(k)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId("641912fa6f22ba679557492f") }
}
ejemplobd>
```

Nota 2: El resultado de la inserción es un objeto con dos campos, el primero acknowledged que indica si el documento fue admitido en la colección y el segundo indica el campo _id correspondiente al identificador único del documento.

Nota 3: si deseamos consultar por los registros almacenados, podemos usar el comando **find**. Esto es,

Sentencia SQL de búsqueda	Sentencia MongoDB de Búsqueda
select * from usuarios	db.usuarios.find()

```
ejemplobd> db.usuarios.find()
[
  {
    _id: ObjectId("641912df6f22ba679557492e"),
    nombre: 'Teresa',
    pais: 'Chile'
  },
  {
    _id: ObjectId("641912fa6f22ba679557492f"),
    nombre: 'Juan',
    pais: 'Chile'
  }
]
ejemplobd>
```

Todos los documentos MongoDB deben tener un campo **_id** con un valor único. Si no lo indicamos en el documento, MongoDB lo crea de manera automática. Mongo crea un valor de **ObjectId** exclusivo para el campo antes de insertarla en la colección.

b) Forma 2:

Insertar **UN documento** a la colección **usuarios** con la función **insert**One({dato, dato, dato,…n}).

1 documento

La instrucción sería: db.**usuarios**.**insertOne**({dato, dato, dato,…n})

```
ejemplobd> db.usuarios.insertOne({nombre:"Sami",pais:"Argentina"})
{
  acknowledged: true,
  insertedId: ObjectId("641a397d6f22ba6795574930")
}
ejemplobd>
```

c) Forma 3:

Insertar varios documentos a la colección **usuarios** con la función **insertMany**([documento, documento, documento, ……n]).

varios documentos

El comando sería: db.**usuarios**.**insertMany**([documento, documento, … n])

```
ejemplobd> db.usuarios.insertMany([{nombre:"pepe",pais:"perú"},{nombre:"coti",pais:"Chile"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("641a3ebe6f22ba6795574931"),
    '1': ObjectId("641a3ebe6f22ba6795574932")
  }
}
ejemplobd>
```

10. Si deseamos limpiar la consola de la Shell podemos usar el comando **cls o** usando las teclas **ctrl +L**

```
mongosh mongodb://127.0.0.1:27017/27017?directConnection=true&serverSelectionTimeoutMS=2000
ejemplobd>
```

11. Si deseamos salir de la consola de la Shell podemos usar el comando **exit o exit().** Además, podemos **usar quit o quit()**

    Cierra la Shell con exit

12. Abre nuevamente la Shell ejecutando mongosh.exe y sigue los siguientes pasos:
    a. Ingresar el puerto 27017

```
C:\Program Files\MongoDB\Server\6.0\bin\mongosh.exe
Please enter a MongoDB connection string (Default: mongodb://localhost/): 27017
```

```
mongosh mongodb://127.0.0.1:27017/27017?directConnection=true&serverSelectionTimeoutMS=2000
Please enter a MongoDB connection string (Default: mongodb://localhost/): 27017
27017
Current Mongosh Log ID: 641a4e454c27839756290e73
Connecting to:          mongodb://127.0.0.1:27017/27017?directConnection=true&serverSelectionTimeoutMS=2000&appName=mong
osh+1.8.0
Using MongoDB:          6.0.5
Using Mongosh:          1.8.0

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

------
   The server generated these startup warnings when booting
   2023-03-16T07:16:32.803-03:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
------

------
   Enable MongoDB's free cloud-based monitoring service, which will then receive and display
   metrics about your deployment (disk utilization, CPU, operation statistics, etc).

   The monitoring data will be available on a MongoDB website with a unique URL accessible to you
   and anyone you share the URL with. MongoDB may use this information to make product
   improvements and to suggest MongoDB products and deployment options to you.

   To enable free monitoring, run the following command: db.enableFreeMonitoring()
   To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
------

27017>
```

    b. Consultar las bases de datos existentes con **show dbs**

```
27017> show dbs
admin       40.00 KiB
config      60.00 KiB
ejemplobd   72.00 KiB
local       72.00 KiB
27017>
```

c. **Activar la base de datos ejemplobd** debe usar el comando **use**

```
27017> use ejemplobd
switched to db ejemplobd
ejemplobd>
```

Ahora podemos crear más colecciones e insertar documentos a las colecciones.

d. **Eliminar todos los documentos** de una **colección** debe usar la instrucción db.nombreColeccion.**deleteMany**({}) donde:

   i. **db** indica la base de datos activa
   ii. nombreColeccion es el nombre de la colección
   iii. **deleteMany()** es el método que permite eliminar todos los documentos de la colección indicada en la instrucción
   iv. **{}** objeto vacío

Nota 1: si se eliminan todos los documentos la colección sigue existiendo.
Nota 2: Posteriormente, veremos la eliminación de algunos documentos que cumplen cierta condición.

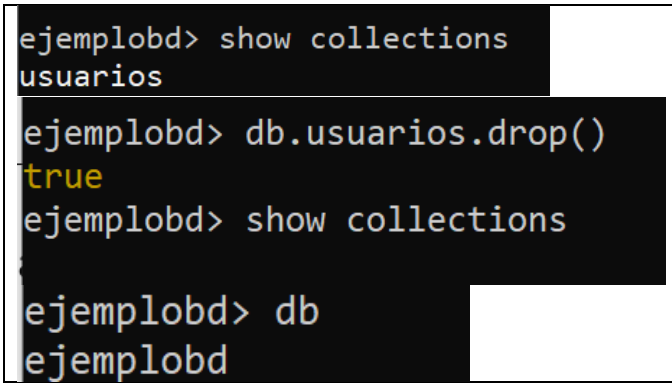
Ejemplo: db.usuarios.**deleteMany**({})

`ejemplobd> show collections`
`usuarios`
`ejemplobd> db.usuarios.deleteMany({})`
`{ acknowledged: true, deletedCount: 5 }`
`ejemplobd>`
`ejemplobd> show collections`
`usuarios`
`ejemplobd> db.usuarios.find()`
`ejemplobd>`
	Primero consultamos por las colecciones existentes
	Segundo, eliminamos los documentos de la colección usuario
	Tercero, validamos que la colección no se ha eliminado
	Cuarto, validamos que no existen documentos en la colección usuario

e. **Eliminar la colección y todos sus documentos** debe usar la instrucción: db.nombreColeccion.drop() donde:

   i. **db** indica la base de datos activa
   ii. nombreColeccion es el nombre de la colección
   iii. **drop()** es el método que permite eliminar todos los documentos de la colección y elimina la colección

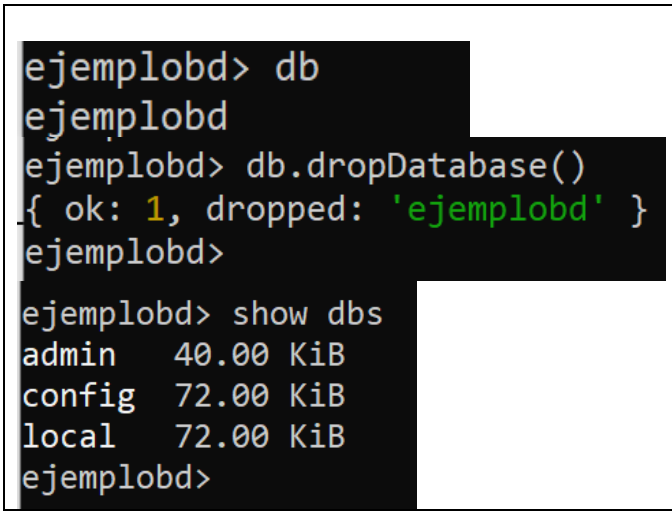
Ejemplo: db.usuarios.**drop()**

```ejemplobd> show collections usuarios ejemplobd> db.usuarios.drop() true ejemplobd> show collections ejemplobd> db ejemplobd``` 	Primero consultamos por las colecciones existentes Segundo, eliminamos la colección usuarios Tercero, validamos que la colección se ha eliminado Cuarto, validamos que la base de datos activa es ejemplobd y sigue existiendo

f. **Eliminar la base de datos con todo su contenido** debe usar la instrucción db.dropDatabase() donde:

   i. **db** indica la base de datos activa
   ii. **dropDatabase()** es el método que permite eliminar la base de datos activa

Ejemplo: db.**dropDatabase()**

```ejemplobd> db ejemplobd ejemplobd> db.dropDatabase() { ok: 1, dropped: 'ejemplobd' } ejemplobd> ejemplobd> show dbs admin 40.00 KiB config 72.00 KiB local 72.00 KiB ejemplobd>``` 	Primero consultamos por la base de datos activa Segundo, eliminamos la base de datos ejemplobd Tercero, validamos que la base de datos ya no existe en el servidor MongoDB